If we could pick an arbitary tree with n nodes, the worst case for BFS would be a star graph: one central node connected to each of the other $n - 1$. Almost all searches in this graph visit and process the center node, which places all others into the queue.

In a grid we cannot do the same thing because our branching factor is limited to a constant. Still, it's a reasonable guess that the optimal solution will somehow resemble the star graph.

Small n

We can start by investigating small n. For those we can build all trees (e.g., incrementally: take all trees of size $n - 1$ and for each try adding one more cell adjacent to exactly one of its cells.) and process each of them to compute its quality.

```
optimal solution for n=9, quality=505
##.##
##.##
.....
##.##
##.##

optimal solution for n=13: quality=1461
##.##
#...#
##.##
.....
.#.##
##.##
```

We see that we can view these maze as starting in the middle and then gradually expanding in all directions.

Medium n

We can get a better idea by continuing greedily: for each n, keep just a small subset of trees with top quality before moving on to generating trees for $n + 1$.

```
greedily constructed good tree for n=138
##########.##########
#########...#########
##########.##########
#######.#...#.#######
#######..#.#..#######
######.#.....#.######
####....##.##....####
#####.#.#...#.#.#####
###..#...#.#...#..###
####..##.....##..####
#.#.#.#.##.##.#.#.#.#
......................
##.#.#.#.#.#.#.#.#.##
##.##...#...#..#..###
######.#.#.#.#..#.###
#######.......#..####
########.#.#.##.#####
#########...#########
##########.##########
########.....########
##########.##########
```

We are starting to get a pretty good idea what's going on: a fractal-like tree that fills the "circle" (in Manhattan metric) with radius r will have most of its nodes at distance r or similar. And whenever the target node for BFS is close to the boundary of the circle and the source node is in a different subtree of the root (i.e., the cell at the center), the BFS will be forced to visit most of the tree.

Some back-of-the-envelope estimates

If we take $r = 63$, the circle contains a bit over 8000 cells, and as we seem to be able to use more than half of the cells within the circle for our tree, that should be roughly enough for a 5000-cell tree.

It should be possible to choose the topology of the tree so that at least half of its cells are leaves or close-to-leaves, and that most leaves are at distance close to r from the root. (Note that some of these leaves can be inside the circle if the path to them twists and turns.) So it might be realistic to place 3000 of the 5000 nodes at distances say $\geq r - 10$ from the root.

If we can do that, we should now have 3000 very good BFS targets. For each of them, by symmetry, around $3 \cdot 5000/4 = 3750$ nodes will be in other subtrees of the root, and thus good start nodes for the BFS. Each of these BFSs should visit most of the tree. If the remaining ones visit roughly a quarter of the tree on average, that should give us roughly what we need to score 100 points.

The estimates we just made were *very* rough, but it's always better to do at least *some* sanity checks in situations like this one.

Simple deterministic trees

There are a few simple but promising strategies we can try based on the above. As a first step, we can try filling as much as possible within a given circle. One simple way do this is to grow the tree from the center

by simply running BFS, with the extra constrain that you cannot enter a cell if it's already adjacent to other previously explored cells. And once we have that, we can let the code run not just for n steps but until the whole circle is filled as much as possible. And then, once we have a tree with too many nodes, we can easily run all the BFSs and store information that will allow us to tell, for each leaf, how the quality decreases if we remove it. This allows us to prune the extra nodes greedily, producing a somewhat better tree. These strategies should give you a bit over 50 points for the task.

Improving the shape

Now we have filled a promising-looking area with some tree, but its shape is nowhere near optimal – we didn't optimize it yet. We know that we should try to have as many leaves and branches as possible far from the root. We can either spend time on thinking about specific fractals and better patterns we could implement, or we can go the way we did when preparing the task: local optimizations of the structure of the tree.

One easy update type is editing the locations of leaves: add a leaf (bonus if a good one), drop a leaf (bonus if a bad one), check whether solution improved.

A more general update type that's still easy to implement: drop any cell (creating two trees), then add any cell that connects the two trees back together (i.e., a cell that is adjacent to exactly one cell of each tree).

Once we implement these types of updates, we can easily wrap them into a simple hill-climbing / simulated annealing shell and let them converge towards trees with a higher quality without the need to think too much.

One additional update type that helped our solutions converge faster is the use of symmetry: while the good trees aren't *entirely* symmetric, the top half and the bottom half are *roughly* mirror images of each other, and sometimes we can speed up the search by replacing one half with the mirror of the other (and adjusting the number of nodes back to 5000 if necessary): if we managed to make a significant improvement in one half, this way it will transfer to the other.